

CEES and SRCC Computing Resources

Mark R. Yoder, Ph.D.

Overview

Scope: Introductory, review of resources, basic HPC-foo

- Summary of Research Computing resources
- Documentation
- Basic HPC concepts
- Transitioning from Mazama to Sherlock
- SLURM Tips (especially in the Sherlock environment)

Organizations and acronyms:

- Stanford School of Earth, Energy, and Environmental Sciences:
 - Also known as: *Earth, SE3, School of Earth*
- CEES: Computational Earth and Environmental Sciences

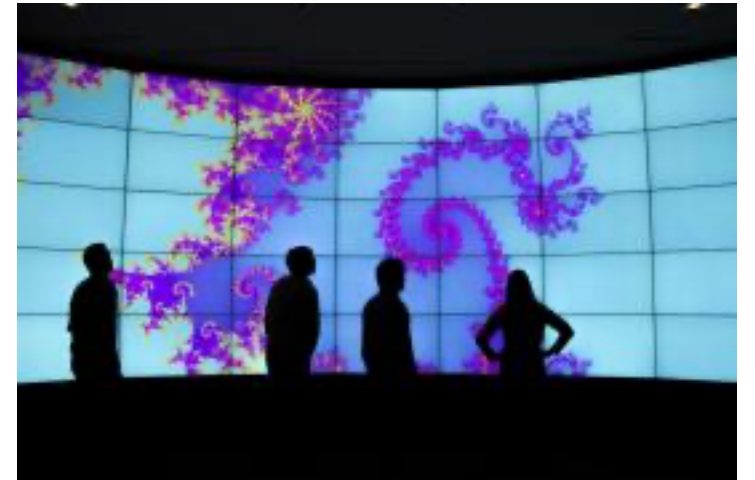
CEES provides a variety of high performance computing (HPC) resources to support the Stanford School of Earth research community
- SRCC: Stanford Research Computing Center

SRCC provides HPC and other research computing support via a number of platforms, including the Sherlock HPC and Oak LUSTRE storage platforms. SRCC also provides NERO OnPrem and Cloud based resources for high risk data, Cloud computing consultation, and a variety of other services related to research computing. Recently, CEES has partnered with SERC to provide expanded and enhanced computing resources to SE3 research teams.
- SERC: “Stanford Earth Research Computing” refers, more or less, directly to *CEES*.

Stanford Research Computing (SRCC)

“Your laptop is not for computing.”

- *“The Stanford Research Computing Center (SRCC) is a joint effort of the Dean of Research and University IT to build and support a comprehensive program to advance research at Stanford.”*
- Provides and manages a wide range of services and facilities
 - Sherlock (general purpose), Nero (high risk), Farmshare (education) HPC
 - Cloud resources and consulting
 - The HIVE visualization facility
 - Compute consulting services
 - Grant and CDMP writing assistance
- If it's about “research” and “computing”, ask...



CEES Resources

- Sherlock *serc* partition (Mostly new! 5456 CPUs, 56 v/a100 GPUs)
 - **104 SH3-CBASE (AMD, 32 cores, 256GB RAM)**
 - **8 SH3-CPERF (AMD, 128 cores, 1TB RAM)**
 - **6 SH3-G86F64 (AMD, 128 cores, 8 x A100 GPU, 1TB RAM)**
 - 12 SH2 base (Intel Skylake, 24 cores, 384 GB RAM)
 - 2 SH2 GPU (Intel Skylake, 24 cores, 4 x V100 GPU)
 - 1PB Oak storage
- Sherlock Public partitions: *normal, dev, bigmem, gpu, owners*
- Mazama
 - ~ 130 active nodes (Intel Haswell, 24 cores, 64 GB)
 - 3 x GPU nodes (8 x V100, 8 x K-80 total GPU devices)
 - 4 tool servers (24 cores, 128, 512GB RAM)
- ~~CEES RCF~~: Recently decommissioned
- FarmShare: Teaching, coursework, extra cycles if you need them

CEES Resources: Comments

- Sherlock:
 - Primary compute platform for most users
 - Single large, *serc* partition shared by all SE3 users; some PIs may have private partitions
 - New workflows should probably start here
- Mazama:
 - Most nodes in (smallish) PI-owned partitions
 - `--partition=cees`: Small (6 ndodes?) shared partition
 - `--partition=twohour`: Default virtual partition; two hour job limits
 - 3 Tool servers (no job scheduling) still available
 - Still quite robust, but nearing retirement
- FarmShare:
 - Teaching cluster. You might use this for classes, or just to get some extra cycles. We won't talk about it here.
- Oak: 1 PB storage

Accounts

- Request accounts via srcc-support@Stanford.edu
- Sherlock, Oak:
 - All Stanford researchers are eligible for an account
 - All Earth researchers:
 - Have access to the *serc* partition
 - Have access to the EES \$OAK space:
 - /oak/stanford/schools/ees
 - (We may need to set up your \$OAK access and PI_group permissions)
- Mazama:
 - All Earth researchers have access to Mazama:
 - HPC: cees, twohour partitions
 - Tool servers
 - /data/cees

Documentation, support

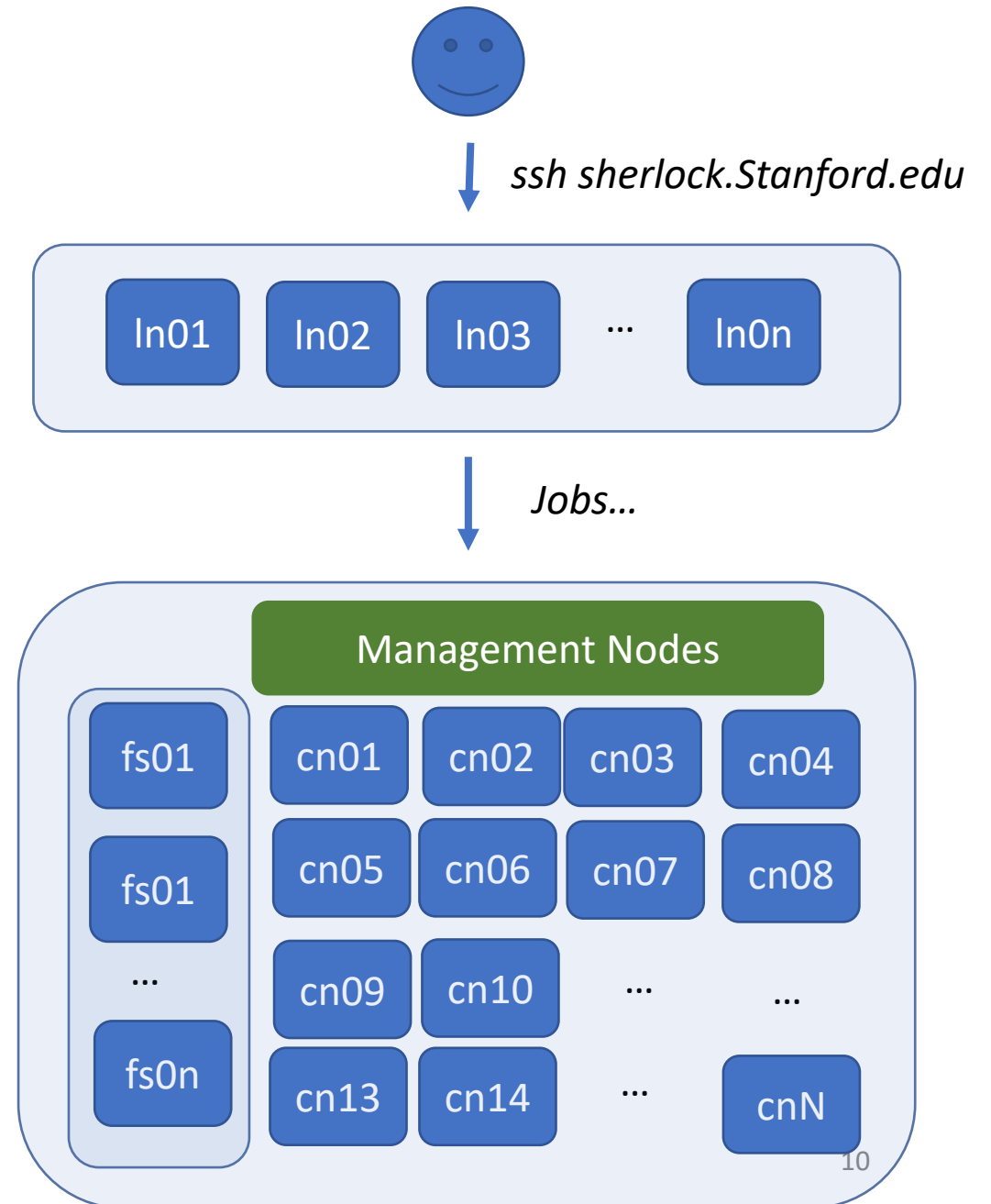
- CEES Documentation: GitHub Pages
 - <https://stanford-rc.github.io/docs-earth/>
 - ** will be moving to something like *cees-docs.stanford.edu*, so standby...
- Sherlock Documentation:
 - <https://www.sherlock.stanford.edu/docs/overview/introduction/>
- NOTE: These docs are searchable!
- Occasional OnBoarding, Basic HPC, and other classes provided by SRCC
- Support requests:
 - CEES Slack channel
 - srcc-support@Stanford.edu

HPC Basics

- What is HPC?
- Basic architecture of HPC
- Nomenclature
- Connecting: ssh, Open on Demand (OoD)
- Running Jobs: Batch, Interactive OoD
- SLURM-foo
- Software on HPC (general), Sherlock
- Filesystems

HPC Basics: What is HPC?

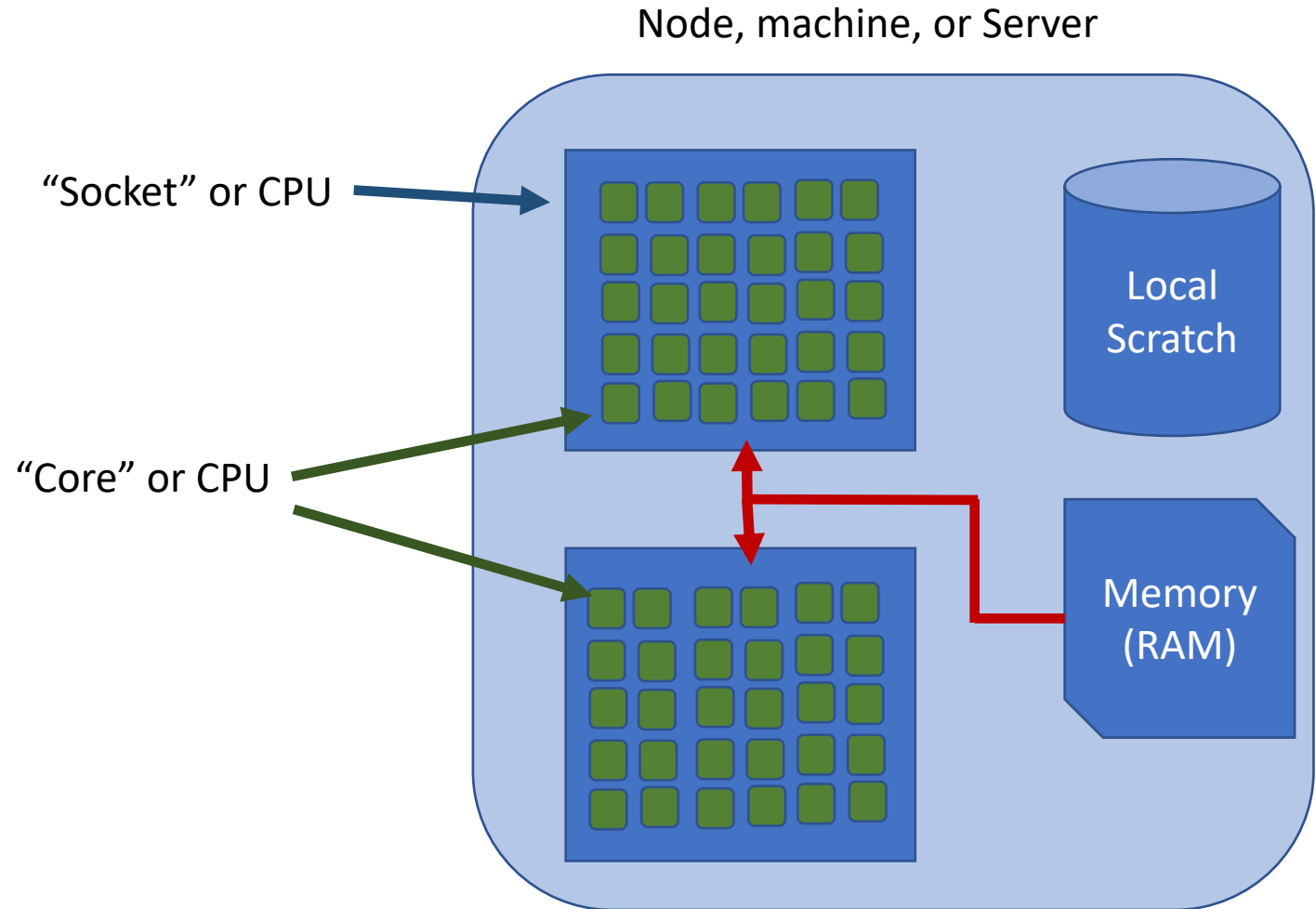
- *High Performance Computing*
- HPC is a bunch of computers:
 - Connected by high speed network
 - Sharing high speed filesystem(s)
 - And some management nodes
- Users access the login API/machines
- Push compute jobs to the compute machines/nodes
- Request resources, to run jobs, via a Job Scheduler



The unfortunate state of HPC nomenclature:

What, exactly, is a “node”?

- The current state of HPC nomenclature is... unfortunate. In SLURM:
- “Node”: A machine; a server; a “computer”
- “Socket”: A traditional “CPU” device.
- Each “socket” or CPU will have many “cores” or CPUs.



HPC Scale: How big is a node?

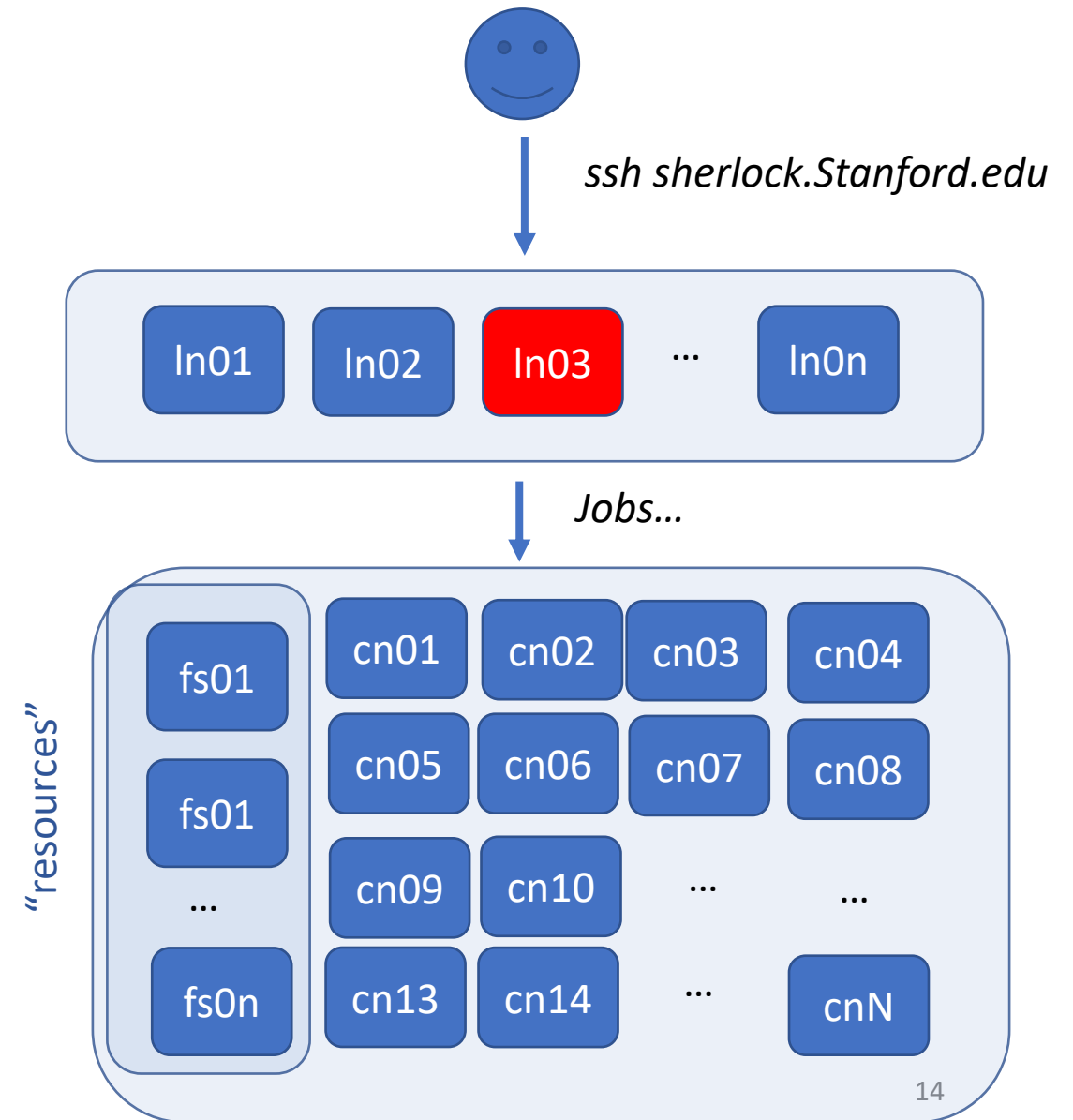
- Sherlock 3.0 CBASE (standard) Node:
 - 1 socket x 32 cores/socket: 32 compute cores (or cpus)
 - 256 GB RAM (8 GB/core or cpu)
- Sherlock 3.0 CPERF (performance) Node:
 - 2 sockets x 64 cores/socket: 128 cpus
 - 1TB RAM (8GB/cpu)
- ~1 or 2 TB local storage (\$LSCRATCH)
- Most filesystems are mounted from designated FS servers.
- As a point of reference:
 - Your laptop (probably): 1 socket, 2-6 CPUs/cores, 8-16 GB RAM.
 - Most NSF HPCs are similar to Sherlock, except ~2GB/core

Connecting to Sherlock (also see docs)

- Connect:
 - SSH: `$ ssh sherlock.stanford.edu`
 - To port graphics `$ssh -XY sherlock.stanford.edu`
 - Homepage: <https://www.sherlock.stanford.edu>
 - OnDemand: <https://login.sherlock.stanford.edu/>
 - 2-factor authentication required
 - Data transfer nodes: `dtn.sherlock.stanford.edu`

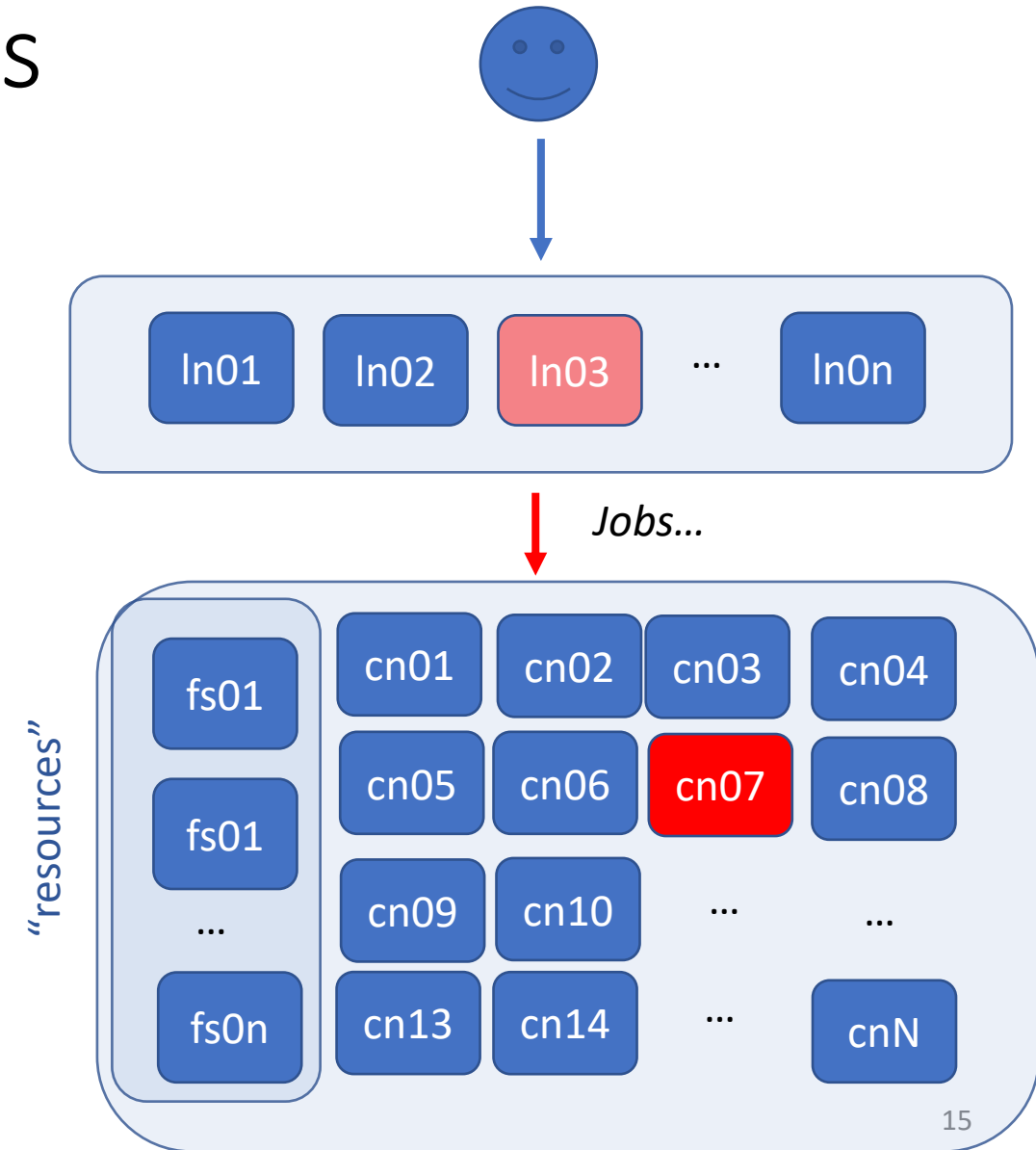
HPC Basics: Login nodes

- Login nodes are **NOT** for computing!
- Connect to HPC, request resources, pass jobs to compute nodes.
- Simple, light weight tasks:
 - Moderate file copy
 - Simple code compiles
 - Very small, lightweight test runs
- Since we own nodes, there is no good reason to run jobs on LN.
- Login nodes:
 - Shared, limited compute capacity, memory, IO, etc. quotas
 - Quota managers will kill jobs
 - Restarted frequently and without warning
 - Therefore, often not a reliable test platform



HPC Basics: Interactive jobs

- Work in real-time
 - Like on the login node or on your laptop
 - But on a compute node
- Commandline/shell, jupyter notebook, R-studio, MatLab, etc.
- Connect to login node, then request resources:
 - `srun -pty -partition=serc,normal bash`
 - `salloc -partition=serc,normal`
 - (You may then need to ssh to allocation)
- Must keep session connected
 - (You can use *screen* to maintain background tasks, but they can be reset)

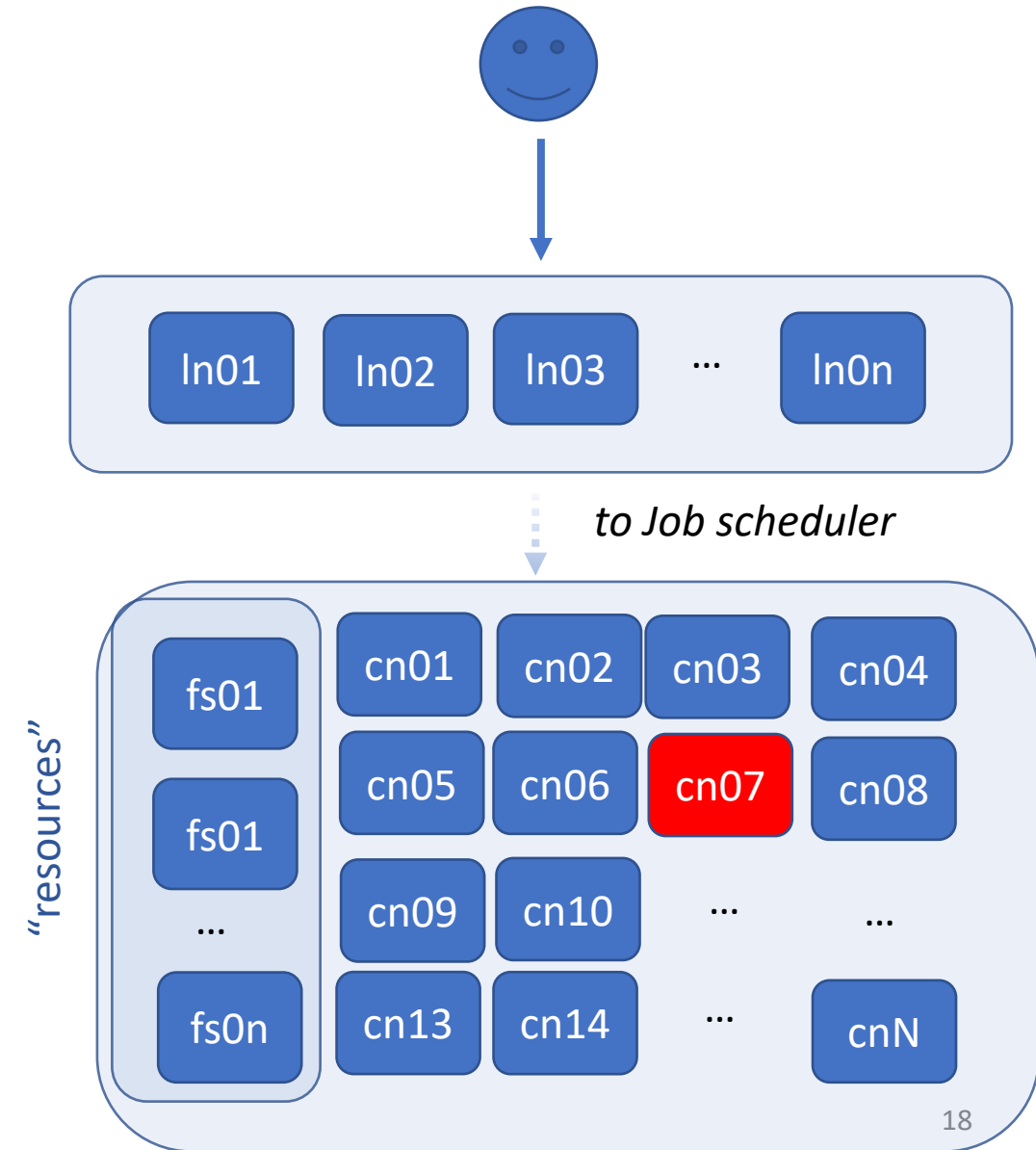


Running Jobs on Sherlock: Interactive

- No tool servers (ie, you can't just log in and run stuff)
- Interactive shell sessions (you can also do this on Mazama):
 - `$ sdev -p serc`
 - `$ srun -pty -partition=serc {other SLURM directives} bash`
 - `$ saloc -partition=serc {other SLURM directives}`
 - Note: *srun bash* and *saloc* are usually interchangeable
- Jupyter notebooks, R-Studio, MatLab from shell sessions or OnDemand (see docs)
- Sherlock OnDemand:
 - Connect: <https://login.sherlock.stanford.edu/>
 - Docs: <https://www.sherlock.stanford.edu/docs/user-guide/ondemand/>

HPC Basics: Batch jobs

- *Preferred modus operandi for HPC!*
- Submit job script to scheduler
- Scheduler will find and allocate resources, then run the job when resources are available
- Runs independently on compute nodes
- Job *std.out* and *std.err* output to file(s)
- Steeper learning curve, but very rewarding.
- Best for big, resource intensive jobs



Sherlock Partitions

- ***Partitions*** are groups of machines, designated a specific purpose or specific users, with distinct access rules.
- ***serc partition is shared by all School of Earth users***
- PI partitions: Some PIs have private partitions on Sherlock
- Public partitions:
 - *normal*: Default partition; heavily subscribed
 - *dev*: Restricted to interactive sessions.
 - *bigmem, gpu*: large memory (4TB) and (public) GPU nodes
- ***owners***:
 - Virtual partition consists of all unassigned resources, available to all owners.
 - Jobs in *owners* will be *preempted* (killed) with a 30 second warning signal
 - Good for short jobs, Monte Carlo type jobs, well checkpointed tasks
 - At last assessment, preemption rate was about 2-3%, more or less time-independent
- **\$OAK storage**:
 - `/oak/stanford/schools/ees/{pi_sunet}`
 - `/oak/stanford/groups/{pi_sunet}`

Running Jobs on Sherlock: Best practices

- Login nodes are not *tool-servers*
- Login nodes are **NOT** for computing!
- Please remember to *srun* and *sbatch* tasks!
- Start your interactive sessions early:
 - Sherlock basically runs flat out 24/7, so “unused” resources are almost always running in *owners*.
 - Because it is a large cluster, with lots of partitions, scheduler can be a little slow
 - It can take 30-60 sec. to schedule a node, even if you own it.

SLURM Requests: Hardware constraints

- SLURM Docs: <https://slurm.schedmd.com/sbatch.html>
- SERC partition includes multiple node configurations and HW architectures
- HW optimized codes, MPI programs, etc. should make HW specific requests using `–constraint= directive(s)`
- To show available constraints:
 - `$ sh_node_feat`
- Examples:
 - `$ sbatch –partition=serc ntasks={n} –constraint=CLASS:SH3_CBASE my_mpi_job.sh`
 - `$ sbatch –partition=serc ntasks={n} –constraint="[CLASS:SH3_CBASE | CLASS:SH3_CPERF]"`
 - `my_mpi_job.sh`
 - `$ sbatch –partition=serc ntasks={n} –constraint=CPU_MNF:AMD my_amd_job.sh`

SLURM Requests: Be specific

- While you're at it...
- Be specific about memory, time, and core/node configuration. Will your request scale?
 - --time=HH:MM:SS
 - --mem-per-cpu, --mem-per-gpu, --mem-per-node (ie, --mem), etc.
 - --cpus-per-gpu
- Default time for interactive sessions is 2 hours
- Unlike Mazama, `--ntasks` , by itself, will likely *not* get you cores on one node (mainly because of *owners*).
- SBATCH docs: <https://slurm.schedmd.com/sbatch.html>

Some SLURM Basics

- `--ntasks`: Parallelization between nodes; independent instances of an app that communicate via something like MPI
- `--ncpus-per-{something}`: Like it sounds. Note that some of these directives can conflict with one another.
- `--mem`: memory *per node*
- `--mem-per-{node, cpu, gpu}`: Determine the memory bottleneck and request memory to scale with that element
- NOTE: Default memory units are MB. Ask for GB (gp, g, G, etc. should all work, depending on SLURM configuration)
 - `--mem-per-cpu=8g`

SLURM Examples (incomplete) for a 256 core job

- MPI program, parallelizes well by messaging; get resources as quickly as possible:
 - `--ntasks=256 --constraint="[CLASS:SH3_CBASE | CLASS:SH3_CPERF]"`
- Only runs via MPI, but benefits from fast message passing:
 - `--ntasks=256 --constraint="[CLASS:SH3_CBASE | CLASS:SH3_CPERF]" --nodes=2-8`
- Benefits well from OMP (threads) parallelization:
 - `--ntasks=8 --cpus-per-task=32 --constraint="[CLASS:SH3_CBASE | CLASS:SH3_CPERF]"`
- Poor inter-node, but good OMP parallelization
 - `--ntasks=2 --cpus-per-task=128 --constraint=CLASS:SH3_CPERF`

More SLURM directive examples...

- Python job, using `multiprocessing`:
 - NOTE: This job is probably not HW sensitive, so consider framing it to use any serc hardware
 - `--ntasks=1 --cpus-per-task=24`
- Most GPU jobs will run on one node, as a single task. Remember, on Sherlock `--partition=gpu` is a public partition with mostly k-80 GPUs
 - `--partition=serc --ntasks=1 --gpus=1 --cpus-per-gpu=8 --mem-per-cpu=8g`
 - NOTE: serc GPUs are 128 cores, 8 GPU (16 cpus/gpu) and 8GB/core, so this example is an under-ask.

Jupyter Notebooks

- Yes, you can run *Jupyter* Notebooks on Sherlock!
- Option 1: ssh and port forwarding (see documentation)
- Option 2: *Sherlock OnDemand* (nominally the preferred method)
 - Web based GUI interface
 - Connect: <https://login.sherlock.stanford.edu/>
 - Docs: <https://www.sherlock.stanford.edu/docs/user-guide/ondemand/>
 - OnDemand has come a LONG way – both on Sherlock and in general, in the past few years, further improvement is expected.

Jupyter notebooks...

- OnDemand will help you define SLURM directives...

Home / My Interactive Sessions / Jupyter Notebook

Interactive Apps

Servers

Jupyter Notebook

RStudio Server

TensorBoard

Jupyter Notebook

This app will launch a [Jupyter Notebook](#) server on [Sherlock](#).

Python version

Python 3.6

Extra Jupyter arguments (optional)

Additional modules (optional)

Space separated list of additional modules to load (eg. `py-numpy/1.17.2_py36`)
Full list at <https://www.sherlock.stanford.edu/docs/software/list>

Partition

dev

Partition to submit the job to

#CPUs

1

Number of CPU cores to allocate

#GPUs (optional)

Number of GPUs to allocate (don't forget to load the `cuda` module in the `Modules` field)

Software on Sherlock

- Sherlock SW stack and modules
 - Uses LMOD, like Mazama
 - Available Modules: `module spider {something}` or `module avail`
- Custom compiles:
 - `$GROUP_HOME` is usually the best place
- CEES SW stack (*beta* release):
 - A bit of a work in progress
 - Built (primarily) using Spack
 - HW, Compiler, MPI consistent toolchains
 - To set up:
 - ~~`./oak/stanford/schools/ees/share/serc_env.sh`~~
 - New SW stacking coming very soon. Tentatively, you can try it out:
 - `./home/groups/s-ees/share/ees/spack_cees/scripts/ees_sw_setup-beta.sh`
 - `module spider gcc-cees/`
 - Revisions coming soon...

Sherlock Filesystems: Flavors, limits, quotas

<https://www.sherlock.stanford.edu/docs/storage/overview/#quotas-and-limits>

- Use: `$ sh_quota`
- `$HOME` (15 GB)
 - `/home/users/{SUNetID}`
 - Small data files, small SW builds, your personal space
- `$GROUP_HOME` (1TB)
 - `/home/groups/{pi SUNetID}`
 - Shared data, specialized SW builds. Secure for your group.
- `$SCRATCH`, `$GROUP_SCRATCH` (100 TB each)
 - `/scratch/users/{SUNetID}`
 - `/scratch/groups{PI SUNetID}`
 - Fast. Temporary (90 day rolling purge). When feasible, do your IO here.
- `$L_SCRATCH`: Local (on-node) scratch; not shared
- `$OAK` (?, 1PB)
 - `/oak/stanford/schools/ees/{PI SUNetID}`
 - `/oak/stanford/groups/{PI SUNetID}` (if your PI has a group space)
 - Most of your data will go here
 - Shared containers and some SW

```
$ sh_quota
+-----+
| Disk usage for user kilian (group: ruthm) |
+-----+
| Filesystem | volume / limit | inodes / limit |
+-----+
| HOME | 9.4GB / 15.0GB [||||| 62%] | - / - ( -%) |
| GROUP_HOME | 562.6GB / 1.0TB [||||| 56%] | - / - ( -%) |
| SCRATCH | 65.0GB / 100.0TB [ 0%] | 143.8K / 50.0M ( 0%) |
| GROUP_SCRATCH | 172.2GB / 100.0TB [ 0%] | 53.4K / 50.0M ( 0%) |
| OAK | 30.8TB / 240.0TB [| 12%] | 6.6M / 36.0M ( 18%) |
+-----+
```

Mazama – Sherlock analogues

Mazama

- FILESYSTEM

- \$HOME
- \$SCRATCH
- \$LOCAL_SCRATCH
- /data, etc...

- *twohour* queue

- Tool-servers

Sherlock:

- FILESYSTEM

- \$HOME
- \$GROUP_HOME
- \$SCRATCH
- \$GROUP_SCRATCH
- \$L_SCRATCH
- \$OAK: PI Group
- \$OAK: CEES

- *owners, serc*, public queues

- Interactive sessions

Summary conclusions

- Lots of new compute resources in Sherlock!
- Mazama (tool servers, HPC) still available
- Sherlock access and support:
 - srcc-support@Stanford.edu
 - CEES support: <https://stanford-rc.github.io/docs-earth/>
 - Sherlock support:
<https://www.sherlock.stanford.edu/docs/overview/introduction/>
 - `$ ssh sherlock.stanford.edu`
 - Use batch and interactive jobs
 - Jupyter Notebooks: *Sherlock OnDemand*, ssh-forwarding